

Name: Krish Agrawal

Roll No: 381041

Batch: A2

Assignment No: 3

Problem Statement:

Fine-tune GPT or GPT-2 for creative story generation.

Objective:

- To understand GPT-2's decoder-only Transformer architecture and the concept of domain-specific fine-tuning.
- To construct a custom story dataset and convert it into a Hugging Face Dataset for training.
- To tokenize the corpus using the GPT-2 tokenizer with fixed-length padding and truncation.
- To run fine-tuning using the Hugging Face Trainer API under a causal language modeling objective.
- To generate story text from the fine-tuned model via temperature sampling and evaluate output quality.

S/W Packages and H/W apparatus used:

- Operating System: Windows/Linux/macOS
- Kernel: Python 3.x
- Tools: Google Colab
- Hardware: GPU (T4/A100 via Colab recommended); CPU minimum 8GB RAM
- Libraries and packages used: Transformers (GPT2Tokenizer, GPT2LMHeadModel, Trainer, TrainingArguments, DataCollatorForLanguageModeling), Datasets, Pandas, Accelerate

Theory:

GPT-2 is an autoregressive language model by OpenAI built on a decoder-only Transformer. It was pre-trained on 40GB of internet text using causal language modelling — predicting each next token given all preceding tokens. The base variant holds 117M parameters, offering a practical balance between capability and computational cost for fine-tuning experiments.

Transfer Learning and Fine-tuning:

Transfer learning adapts a model pre-trained on a broad corpus to a narrower target domain by continuing training on domain-specific data. For GPT-2, all weights are updated via gradient descent on the custom corpus. The pre-trained initialisation encodes general linguistic patterns, while fine-tuning shifts the generation distribution towards the target style — here, creative short-story narration.

Causal Language Modelling (CLM):

CLM trains the model to predict the next token in a strictly left-to-right fashion, never attending to future tokens. DataCollatorForLanguageModeling with `mlm=False` configures this objective by auto-shifting labels one position ahead of inputs. This makes GPT-2 inherently suited for open-ended text generation where future tokens are unknown at inference time.

Hugging Face Trainer API:

The Trainer abstraction manages the full training loop — batching, gradient accumulation, mixed precision, checkpointing, and logging — through a unified interface. TrainingArguments centralises hyperparameter specification including epochs, batch size, logging steps, and output directory, significantly reducing the boilerplate required to run fine-tuning experiments.

Text Generation with Sampling:

After fine-tuning, model.generate() produces text by iteratively sampling the next token. The temperature parameter scales the logit distribution — higher values increase creativity at the cost of coherence. Top-k sampling restricts candidates to the k highest-probability tokens, preventing implausible word selections. Generated token IDs are decoded back to human-readable text by the tokenizer.

Methodology:

- Install transformers, datasets, and accelerate libraries.
- Import GPT2Tokenizer, GPT2LMHeadModel, Trainer, TrainingArguments, DataCollatorForLanguageModeling, and Dataset.
- Create a five-sentence story dataset spanning fantasy, sci-fi, mystery, and adventure; store as a Pandas DataFrame and convert to a Hugging Face Dataset.
- Load the GPT-2 tokenizer; assign the EOS token as pad token (GPT-2 has no native pad token).
- Load GPT2LMHeadModel from the 'gpt2' checkpoint.
- Define a tokenize_function with truncation=True, padding='max_length', max_length=128 and map it over the dataset.
- Initialise DataCollatorForLanguageModeling with mlm=False for CLM data preparation.
- Configure TrainingArguments: 5 epochs, batch size 2, output_dir='./gpt2-story', logging every 50 steps.
- Instantiate and call Trainer.train(); after training generate a story from 'Once upon a time' with max_length=100, temperature=0.9, top_k=50.
- Save the model and tokenizer via save_pretrained() for future reuse.

Use Cases:

- Use Case 1 - Children's Story Generation: Fine-tuning on fairy tales yields age-appropriate narrative generators for interactive educational apps and digital storytelling platforms.
- Use Case 2 - Game Dialogue Production: Studios fine-tune on quest documents and NPC scripts to auto-generate contextually consistent in-game text, reducing manual content authoring effort.
- Use Case 3 - Personalised Writing Assistants: Writers fine-tune on their own prose to create a style-matched continuation generator that preserves their vocabulary and narrative voice.

Advantages:

- Pre-trained weights provide strong linguistic foundations, needing only a small domain corpus to shift style.
- The Trainer API removes the need for custom PyTorch training loops.
- CLM objective suits open-ended generation without requiring label annotations.
- Temperature and top-k parameters offer controllable creativity-coherence trade-offs.
- Saved checkpoints allow reuse without repeating the fine-tuning process.

Limitations:

- Five training sentences are far too few; hundreds to thousands of stories are needed for genuine quality.
- GPT-2 lacks explicit story structure memory, struggling to maintain consistent plot, characters, or setting.
- Training on a tiny dataset risks overfitting rather than generalising narrative patterns.
- The 117M-parameter base model is modest; larger variants produce significantly more fluent long-form text.
- CPU fine-tuning is impractically slow; GPU access is essential for reasonable training times.

Applications:

- On-demand personalised story and poem generation for creative writing platforms.
- Dynamic branching narrative engines for interactive fiction and adventure games.
- Brand-voice-matched copywriting for marketing campaigns and product descriptions.
- Scriptwriting support for scene descriptions, monologues, and plot outlines.
- Curriculum-aligned educational storytelling for vocabulary and moral lesson delivery.

Conclusion:

This assignment fine-tuned GPT-2 for creative story generation using the Hugging Face Transformers library. A five-sentence dataset was tokenized to 128-token fixed-length sequences and used for 5-epoch CLM fine-tuning via the Trainer API. The fine-tuned model generated a story continuation from 'Once upon a time' using temperature-based top-k sampling. While the small corpus constrains generation quality, the experiment successfully demonstrates the complete fine-tuning workflow from data preparation to inference and model persistence. Expanding the training corpus with diverse, well-structured stories would substantially improve narrative fluency and creative variety.